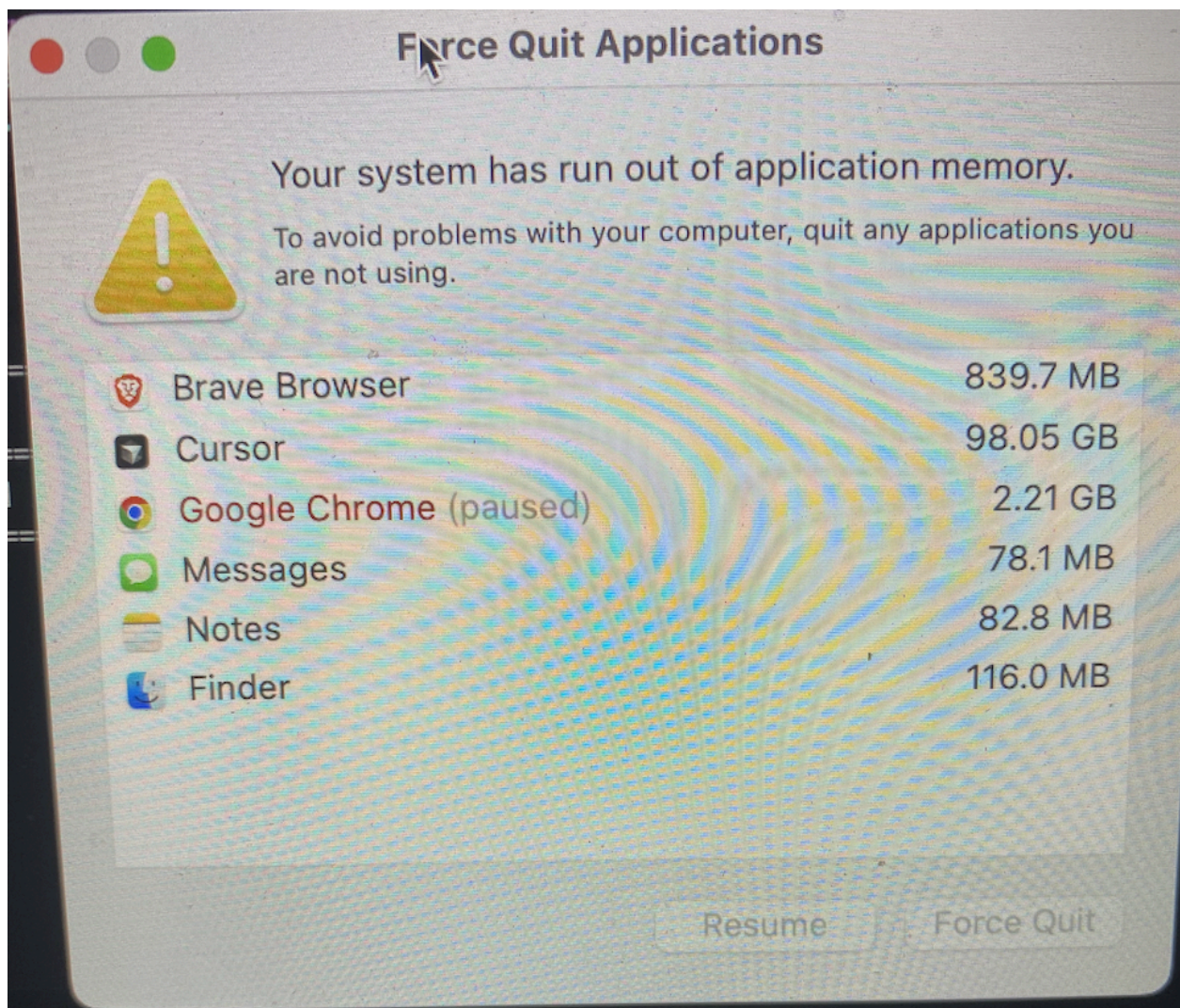
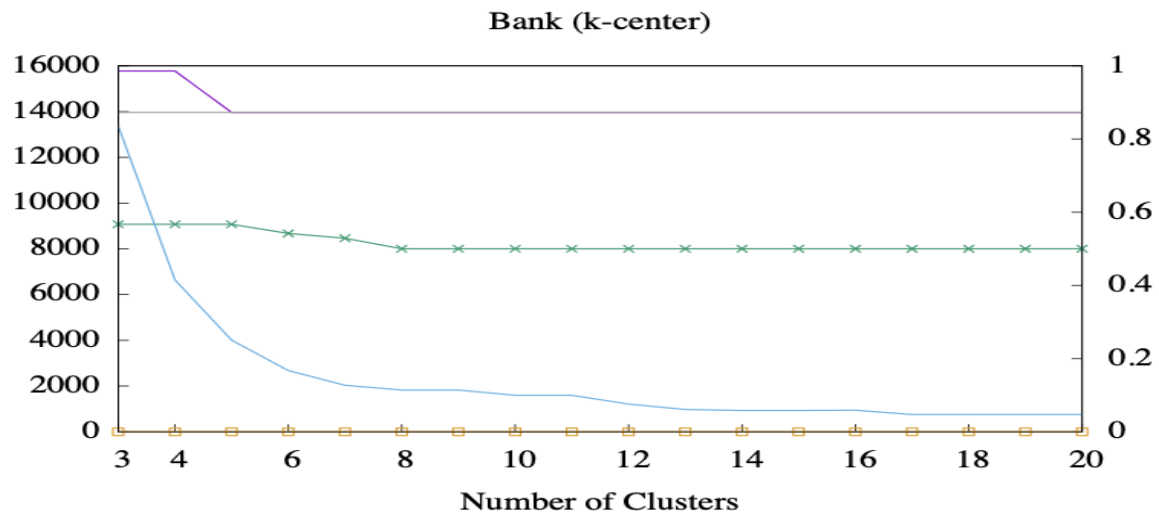




```
capacitated_costs) if not np.isnan(c)]  
(c)]
```

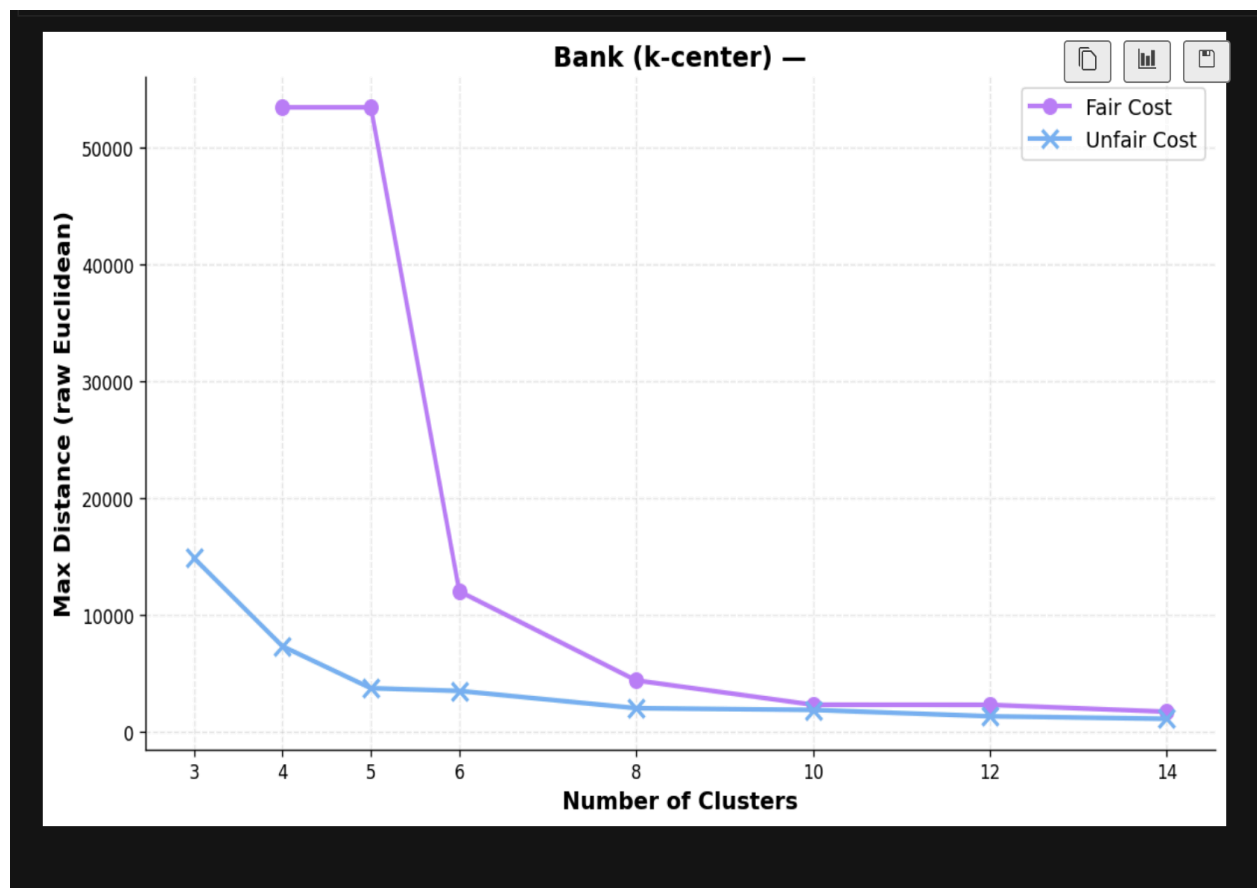
```
width=2.5,
```

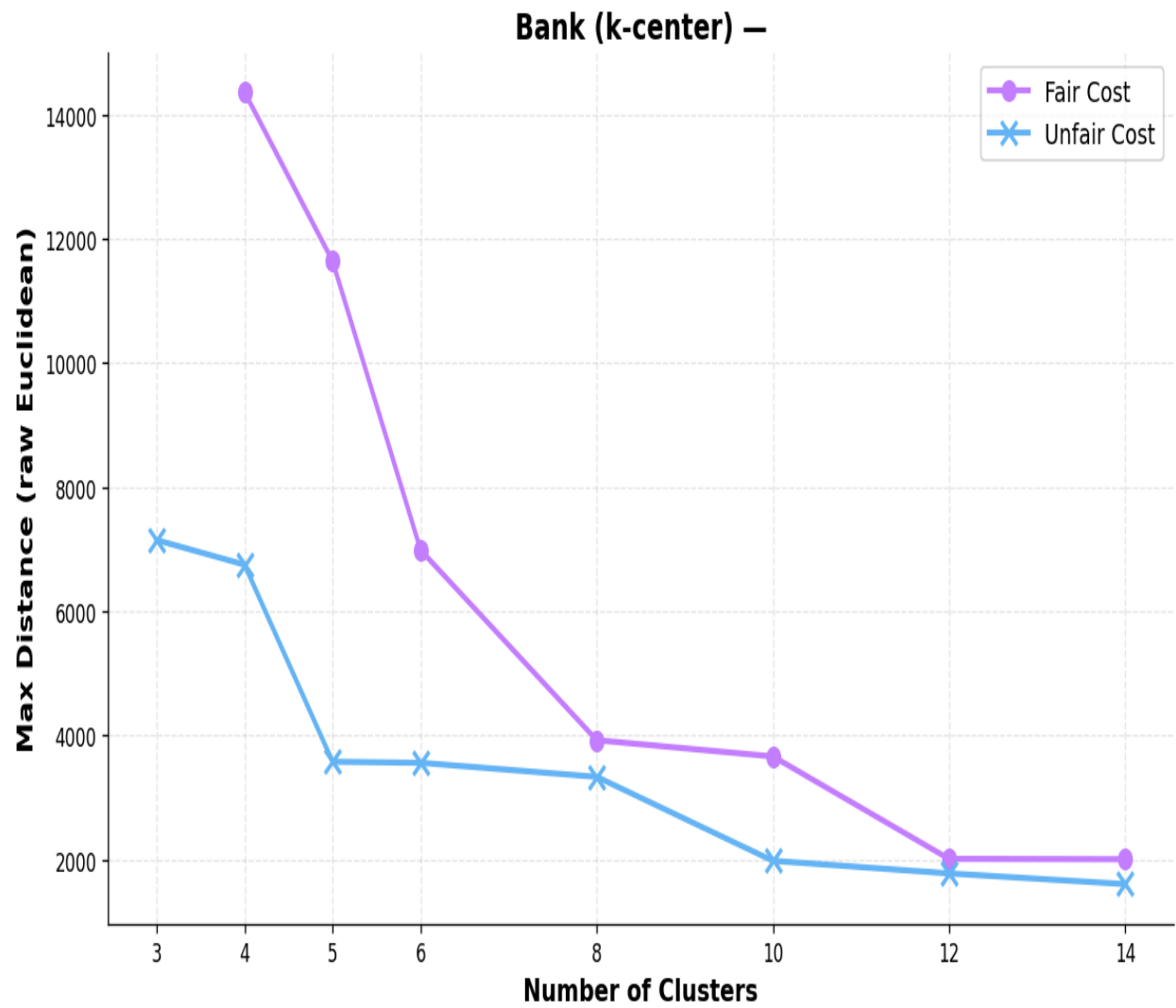
```
es, uncapacitated_costs) if not np.isnan(c)]  
isnan(c)]
```



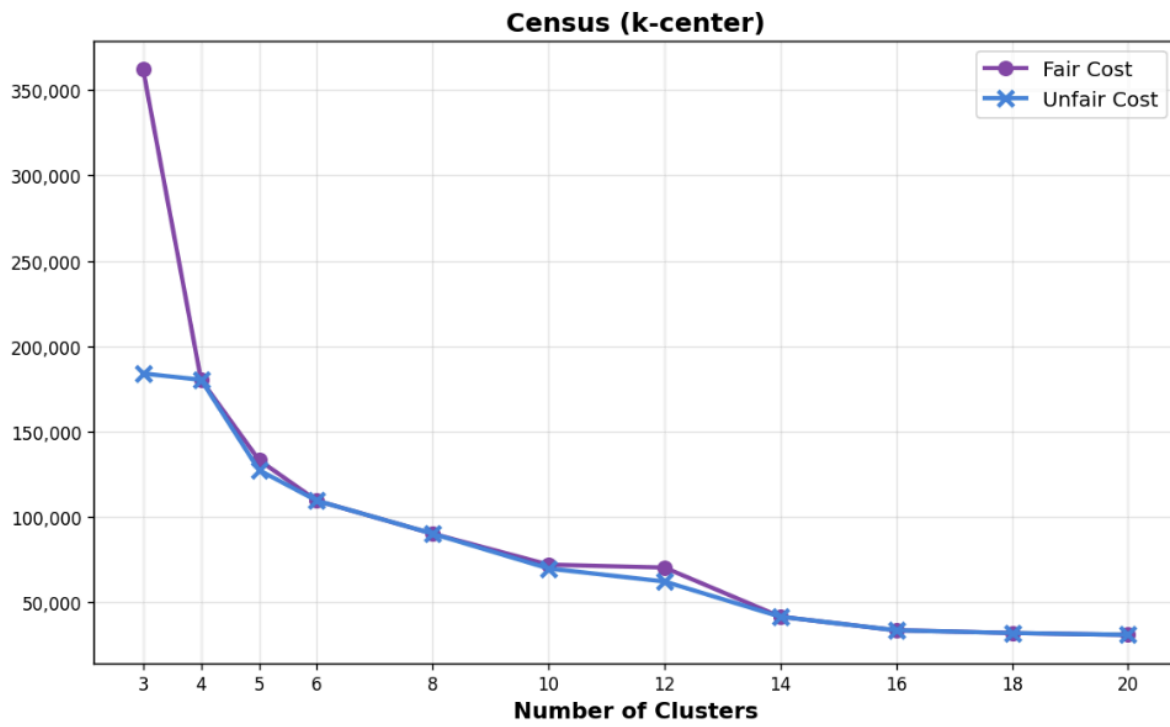
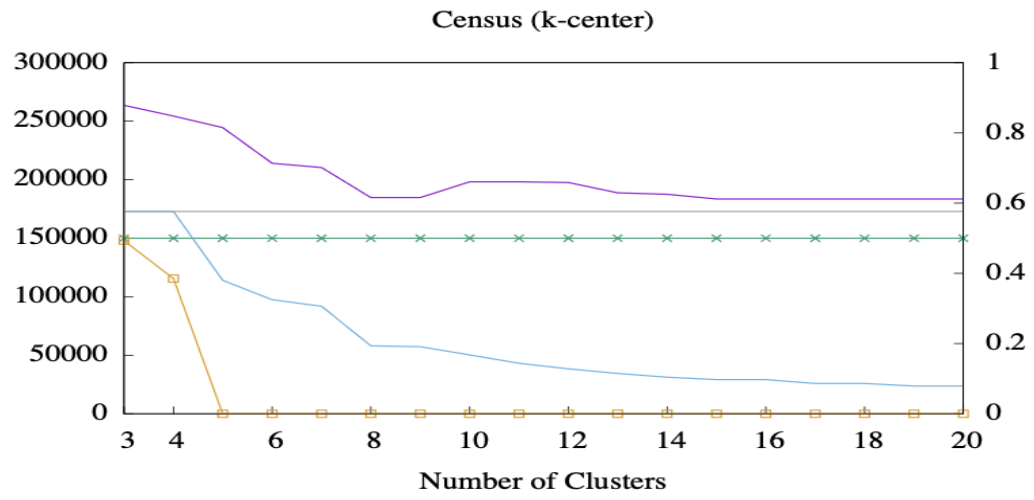
Random seed =42

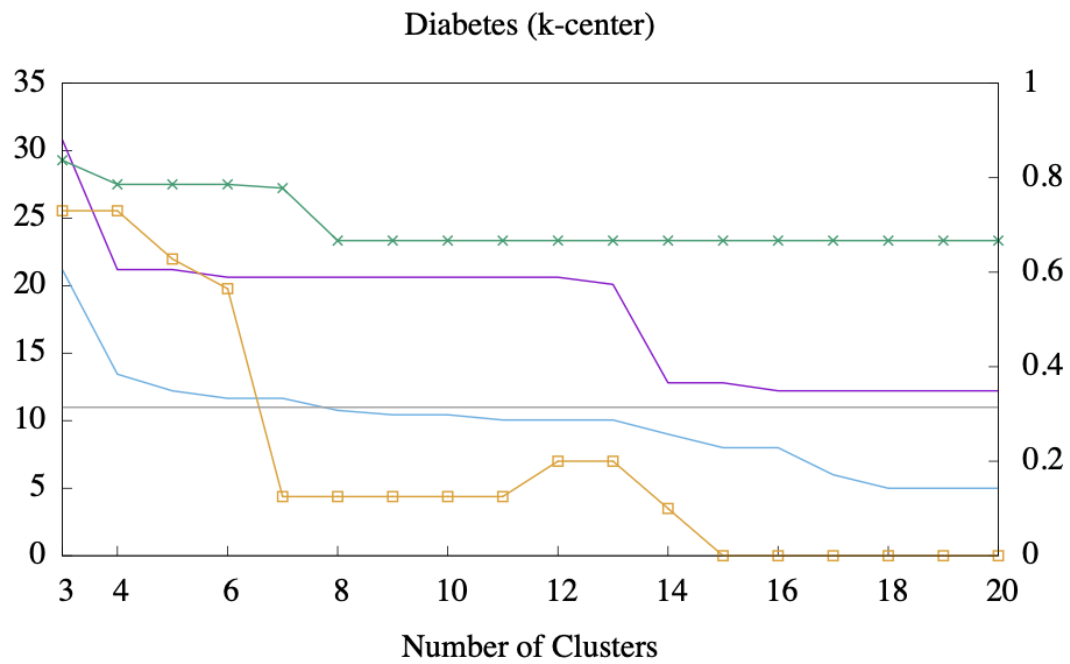
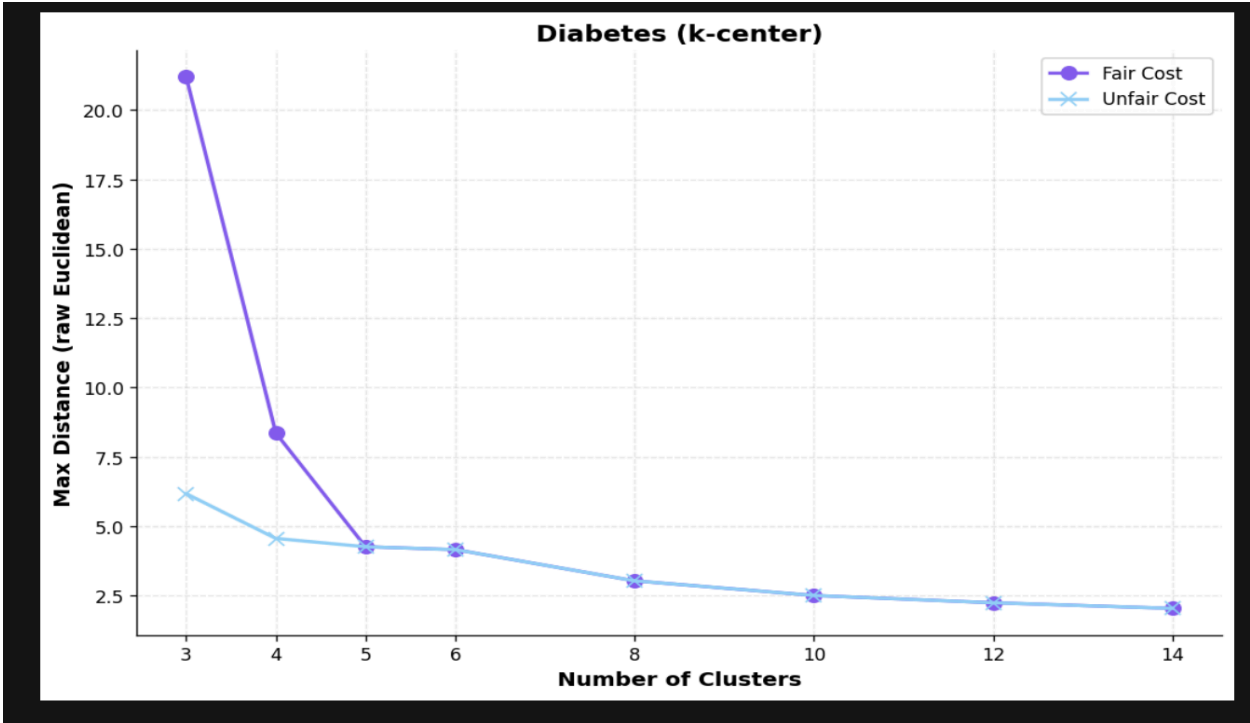
250





Random seed = 4339





Optimization : Spatial Indexing for Lines 12–14 (kills the 100GB)

Lines 12–13 compute:

- $C_i \leftarrow \{c \in C \mid d(c, f_i) \leq r + r'\}$
- $F_i \leftarrow \{f \in F \mid d(f, f_i) \leq r'\}$

Your current code materializes the **full distance matrix** upfront to answer these queries. For 50K clients, that's $50K \times 50 \times 8 \text{ bytes} \approx 20\text{MB}$ for `dist_CF` alone (manageable), but if you're still computing `dist_CC` anywhere, that's $50K \times 50K \times 8 \text{ bytes} = \mathbf{20GB}$. With multiple copies, intermediate structures, and the flow network, you hit 100GB.

Replace with a KD-tree / BallTree:

A **KD-tree** is a **binary tree used to organize points in k-dimensional space** (like 2D, 3D, etc.) so that we can do **fast nearest neighbor searches**.

Instead of scanning all points ($O(n)$), it helps reduce search time to $\sim O(\log n)$ (on average).

How it Works (Intuition)

Let's say you have 2D points (x, y) .

Step-by-step:

1. Pick a dimension to split (alternate: $x \rightarrow y \rightarrow x \rightarrow y \dots$)
2. Choose a **median point** along that dimension
3. Split space into two halves:
 - Left subtree $\rightarrow$ smaller values
 - Right subtree $\rightarrow$ larger values
4. Repeat recursively

python

```
Python
from sklearn.neighbors import BallTree
```

```

import numpy as np

# Build once (Line 0, before loops)
client_tree = BallTree(np.radians(C), metric='haversine')
facility_tree = BallTree(np.radians(F), metric='haversine')

# Inside Line 11 loop – replaces Lines 12-13
def get_eligible_sets(f_i_prime, r, r_prime, client_tree,
facility_tree):
    # C'_i: clients within r + r' of f'_i (Line 12)
    C_i = client_tree.query_radius(
        np.radians([f_i_prime]), r=(r + r_prime) / 6371.0
    )[0]

    # F'_i: facilities within r' of f'_i (Line 13)
    F_i = facility_tree.query_radius(
        np.radians([f_i_prime]), r=r_prime / 6371.0
    )[0]

    return C_i, F_i

```

Impact: Memory drops from $O(|C|^2)$ or $O(|C| \times |F|)$ stored matrix to $O(|C| + |F|)$ for the trees. Query time is $O(\log n)$ per facility center. **This alone can take you from 100GB to under 1GB.**

Line 14 (edge creation) also benefits — you only create edges between eligible pairs returned by the radius queries, not by scanning the full matrix.
